5.3. Ensemble Models: Random Forest

So far in this project, we've learned how to retrieve and decompress data, and how to manage imbalanced data to build a decision-tree model.

In this lesson, we're going to expand our decision tree model into an entire forest (an example of something called an **ensemble model**); learn how to use a **grid search** to tune hyperparameters; and create a function that loads data and a pre-trained model, and uses that model to generate a Series of predictions.

In [2]:
```
import gzip
import json
import pickle

import matplotlib.pyplot as plt
import pandas as pd
from imblearn.over_sampling import RandomOverSampler
from IPython.display import VimeoVideo
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
```

In [3]:
```
VimeoVideo("694695674", h="538b4d2725", width=600)
```

Out[3]:

05:57

Prepare Data

As always, we'll begin by importing the dataset.

Import

Task 5.3.1: Complete the `wrangle` function below using the code you developed in the lesson 5.1. Then use it to import `poland-bankruptcy-data-2009.json.gz` into the DataFrame `df`.

- Write a function in Python.

In [87]:
```
import pandas as pd
import json
import gzip

def wrangle(filename):
    with gzip.open(filename, "rt", encoding="utf-8") as f:
        raw = json.load(f)

    df = pd.DataFrame(raw["data"])
    return df
```

In [88]:
```
df = wrangle("data/poland-bankruptcy-data-2009.json.gz")

print(df.shape)
df.head()
```

```
(9977, 66)
```

Out[88]:

	company_id	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8	fe
0	1	0.174190	0.41299	0.14371	1.3480	-28.9820	0.60383	0.219460	1.12250	1.
1	2	0.146240	0.46038	0.28230	1.6294	2.5952	0.00000	0.171850	1.17210	1.
2	3	0.000595	0.22612	0.48839	3.1599	84.8740	0.19114	0.004572	2.98810	1.
3	5	0.188290	0.41504	0.34231	1.9279	-58.2740	0.00000	0.233580	1.40940	1.
4	6	0.182060	0.55615	0.32191	1.6045	16.3140	0.00000	0.182060	0.79808	1.

5 rows × 66 columns

Split

Task 5.3.2: Create your feature matrix `X` and target vector `y`. Your target is `"bankrupt"`.

- What's a feature matrix?
- What's a target vector?
- Subset a DataFrame by selecting one or more columns in pandas.
- Select a Series from a DataFrame in pandas.

In [ ]:
```
# target = "bankrupt"
# X = ...
# y = ...

# print("X shape:", X.shape)
# print("y shape:", y.shape)
```

In [89]:
```
target = "bankrupt"

X = df.drop(columns=target)
y = df[target]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (9977, 65)
y shape: (9977,)
```

Since we're not working with time series data, we're going to randomly divide our dataset into training and test sets — just like we did in project 4.

Task 5.3.3: Divide your data (`X` and `y`) into training and test sets using a randomized train-test split. Your test set should be 20% of your total data. And don't forget to set a `random_state` for reproducibility.

- Perform a randomized train-test split using scikit-learn.

In [ ]:
```
# X_train, X_test, y_train, y_test = ...


# print("X_train shape:", X_train.shape)
# print("y_train shape:", y_train.shape)
# print("X_test shape:", X_test.shape)
# print("y_test shape:", y_test.shape)
```

In [90]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (7981, 65)
y_train shape: (7981,)
X_test shape: (1996, 65)
y_test shape: (1996,)
```

You might have noticed that we didn't create a validation set, even though we're planning on tuning our model's hyperparameters in this lesson. That's because we're going to use

cross-validation, which we'll talk about more later on.

Resample

In [22]:
```
VimeoVideo("694695662", h="dc60d76861", width=600)
```

Out[22]:

Task 5.3.4: Create a new feature matrix `X_train_over` and target vector `y_train_over` by performing random over-sampling on the training data.

- What is over-sampling?
- Perform random over-sampling using imbalanced-learn.

In [ ]:
```
# over_sampler = ...
# X_train_over, y_train_over = ...
# print("X_train_over shape:", X_train_over.shape)
# X_train_over.head()
```

In [91]:
```
from imblearn.over_sampling import RandomOverSampler

over_sampler = RandomOverSampler(random_state=42)

X_train_over, y_train_over = over_sampler.fit_resample(X_train, y_train)

print("X_train_over shape:", X_train_over.shape)
X_train_over.head()
```

```
X_train_over shape: (15194, 65)
```

Out[91]:

	company_id	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat
0	5796	0.279320	0.053105	0.852030	17.0440	199.080	0.741770	0.353570	16.006
1	1353	0.001871	0.735120	0.156460	1.2269	-10.837	0.000000	0.002938	0.360
2	2190	0.113940	0.490250	0.077121	1.2332	-43.184	-0.000171	0.113940	1.039
3	6667	0.008136	0.652610	0.148120	1.2628	29.071	0.000000	0.008136	0.532
4	5497	0.045396	0.279640	0.708730	3.7656	238.120	0.000000	0.056710	2.576

5 rows × 65 columns

Build Model

Now that we have our data set up the right way, we can build the model. 🏗️

Baseline

Task 5.3.5: Calculate the baseline accuracy score for your model.

- What's accuracy score?
- Aggregate data in a Series using `value_counts` in pandas.

In [ ]:
```
# acc_baseline = ...
# print("Baseline Accuracy:", round(acc_baseline, 4))
```

In [92]:
```
from collections import Counter

# Find the most common class in the training data
majority_class_count = Counter(y_train).most_common(1)[0][1]
total = len(y_train)

acc_baseline = majority_class_count / total

print("Baseline Accuracy:", round(acc_baseline, 4))
```

```
Baseline Accuracy: 0.9519
```

Iterate

So far, we've built single models that predict a single outcome. That's definitely a useful way to predict the future, but what if the one model we built isn't the *right* one? If we could somehow use more than one model simultaneously, we'd have a more trustworthy prediction.

Ensemble models work by building multiple models on random subsets of the same data, and then comparing their predictions to make a final prediction. Since we used a decision tree in the last lesson, we're going to create an ensemble of trees here. This type of model is called a **random forest**.

We'll start by creating a pipeline to streamline our workflow.

In [25]:
```
VimeoVideo("694695643", h="32c3d5b1ed", width=600)
```

Out[25]:

08:01

Task 5.3.6: Create a pipeline named `clf` (short for "classifier") that contains a `SimpleImputer` transformer and a `RandomForestClassifier` predictor. Keep the `random_state = 42` for `RandomForestClassifier` predictor.

- What's an ensemble model?
- What's a random forest model?

In [ ]:
```
# clf = ...
# print(clf)
```

In [93]:
```
clf = make_pipeline(SimpleImputer(), RandomForestClassifier(random_state=42))
print(clf)
```

```
Pipeline(steps=[('simpleimputer', SimpleImputer()),
                ('randomforestclassifier',
                 RandomForestClassifier(random_state=42))])
```

By default, the number of trees in our forest (`n_estimators`) is set to 100. That means when we train this classifier, we'll be fitting 100 trees. While it will take longer to train, it will hopefully lead to better performance.

In order to get the best performance from our model, we need to tune its hyperparameter. But how can we do this if we haven't created a validation set? The answer is **cross-**

validation. So, before we look at hyperparameters, let's see how cross-validation works with the classifier we just built.

In [27]:
```
VimeoVideo("694695619", h="2c41dca371", width=600)
```

Out[27]:

08:15

Task 5.3.7: Perform cross-validation with your classifier, using the over-sampled training data. We want five folds, so set `cv` to 5. We also want to speed up training, to set `n_jobs` to -1.

- What's cross-validation?
- Perform k-fold cross-validation on a model in scikit-learn.

In [94]:
```
cv_acc_scores = cross_val_score(clf, X_train_over, y_train_over, cv=5, n_jobs=-1)
print(cv_acc_scores)
```

```
[1. 1. 1. 1. 1.]
```

That took kind of a long time, but we just trained 500 random forest classifiers (100 jobs x 5 folds). No wonder it takes so long!

Pro tip: even though `cross_val_score` is useful for getting an idea of how cross-validation works, you'll rarely use it. Instead, most people include a `cv` argument when they do a hyperparameter search.

Now that we have an idea of how cross-validation works, let's tune our model. The first step is creating a range of hyperparameters that we want to evaluate.

In [45]:
```
VimeoVideo("694695593", h="5143f0b63f", width=600)
```

Out[45]:

05:57

Task 5.3.8: Create a dictionary with the range of hyperparameters that we want to evaluate for our classifier.

1. For the `SimpleImputer`, try both the `"mean"` and `"median"` strategies.
2. For the `RandomForestClassifier`, try `max_depth` settings between 10 and 50, by steps of 10.
3. Also for the `RandomForestClassifier`, try `n_estimators` settings between 25 and 100 by steps of 25.

- What's a dictionary?
- What's a hyperparameter?
- Create a range in Python
- Define a hyperparameter grid for model tuning in scikit-learn.

In [59]:
```
params = {
    "simpleimputer__strategy": ["mean", "median"],
    "randomforestclassifier__n_estimators": range(25, 100, 25),
    "randomforestclassifier__max_depth": range(10, 50, 10)
}
params
```

Out[59]:
```
{'simpleimputer__strategy': ['mean', 'median'],
 'randomforestclassifier__n_estimators': range(25, 100, 25),
 'randomforestclassifier__max_depth': range(10, 50, 10)}
```

Now that we have our hyperparameter grid, let's incorporate it into a **grid search.**

In [47]:
```
VimeoVideo("694695574", h="8588bf015f", width=600)
```

Out[47]:

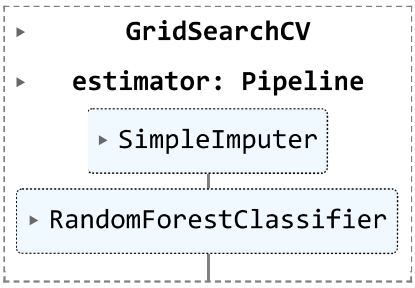
06:07

Task 5.3.9: Create a `GridSearchCV` named `model` that includes your classifier and hyperparameter grid. Be sure to use the same arguments for `cv` and `n_jobs` that you used above, and set `verbose` to 1.

- What's cross-validation?
- What's a grid search?
- Perform a hyperparameter grid search in scikit-learn.

In [60]:
```
model = GridSearchCV(
    clf,
    param_grid=params,
    cv=5,
    n_jobs=1,
    verbose=1
)
model
```

Out[60]:



Finally, now let's fit the model.

In [49]:
```
VimeoVideo("694695566", h="f4e9910a9e", width=600)
```

Out[49]:

02:25

Task 5.3.10: Fit `model` to the over-sampled training data.

In [ ]:
```
# Train model
model.fit(X_train_over, y_train_over)
```

```
Fitting 5 folds for each of 24 candidates, totalling 120 fits
```

This will take some time to train, so let's take a moment to think about why. How many forests did we just test? 4 different `max_depth`s times 3 `n_estimator`s times 2 imputation strategies... that makes 24 forests. How many fits did we just do? 24 forests times 5 folds is 120. And remember that each forest is comprised of 25-75 trees, which works out to *at least* 3,000 trees. So it's computationally expensive!

Okay, now that we've tested all those models, let's take a look at the results.

In [53]:
```
VimeoVideo("694695546", h="4ae60129c4", width=600)
```

Out[53]:

02:57

Task 5.3.11: Extract the cross-validation results from `model` and load them into a DataFrame named `cv_results`.

- Get cross-validation results from a hyperparameter search in scikit-learn.

In [ ]:
```
cv_results = pd.DataFrame(model.cv_results_)
cv_results.head(10)
```

In addition to the accuracy scores for all the different models we tried during our grid search, we can see how long it took each model to train. Let's take a closer look at how different hyperparameter settings affect training time.

First, we'll look at `n_estimators`. Our grid search evaluated this hyperparameter for various `max_depth` settings, but let's only look at models where `max_depth` equals 10.

In [63]:
```
VimeoVideo("694695537", h="e460435664", width=600)
```

Out[63]:

05:42

Task 5.3.12: Create a mask for `cv_results` for rows where `"param_randomforestclassifier__max_depth"` equals 10. Then plot `"param_randomforestclassifier__n_estimators"` on the x-axis and `"mean_fit_time"` on the y-axis. Don't forget to label your axes and include a title.

- Subset a DataFrame with a mask using pandas.
- Create a line plot in Matplotlib.

In [ ]:
```
mask = cv_results["param_randomforestclassifier__maxdepth"] == 10
cv_results[mask]
```

In [ ]:
```
# Create mask
mask = cv_results["param_randomforestclassifier__max_depth"] == 10

# Plot fit time vs n_estimators
plt.plot(
    cv_results[mask]["param_randomforestclassifier__n_estimators"],
    cv_results[mask]["mean_fit_time"]
)

# Label axes
plt.xlabel("Number of Estimators")
plt.ylabel("Mean Fit Time [seconds]")
plt.title("Training Time vs Estimators (max_depth=10)");
```

Next, we'll look at `max_depth`. Here, we'll also limit our data to rows where `n_estimators` equals 25.

In [66]:
```
VimeoVideo("694695525", h="99f2dfc9eb", width=600)
```

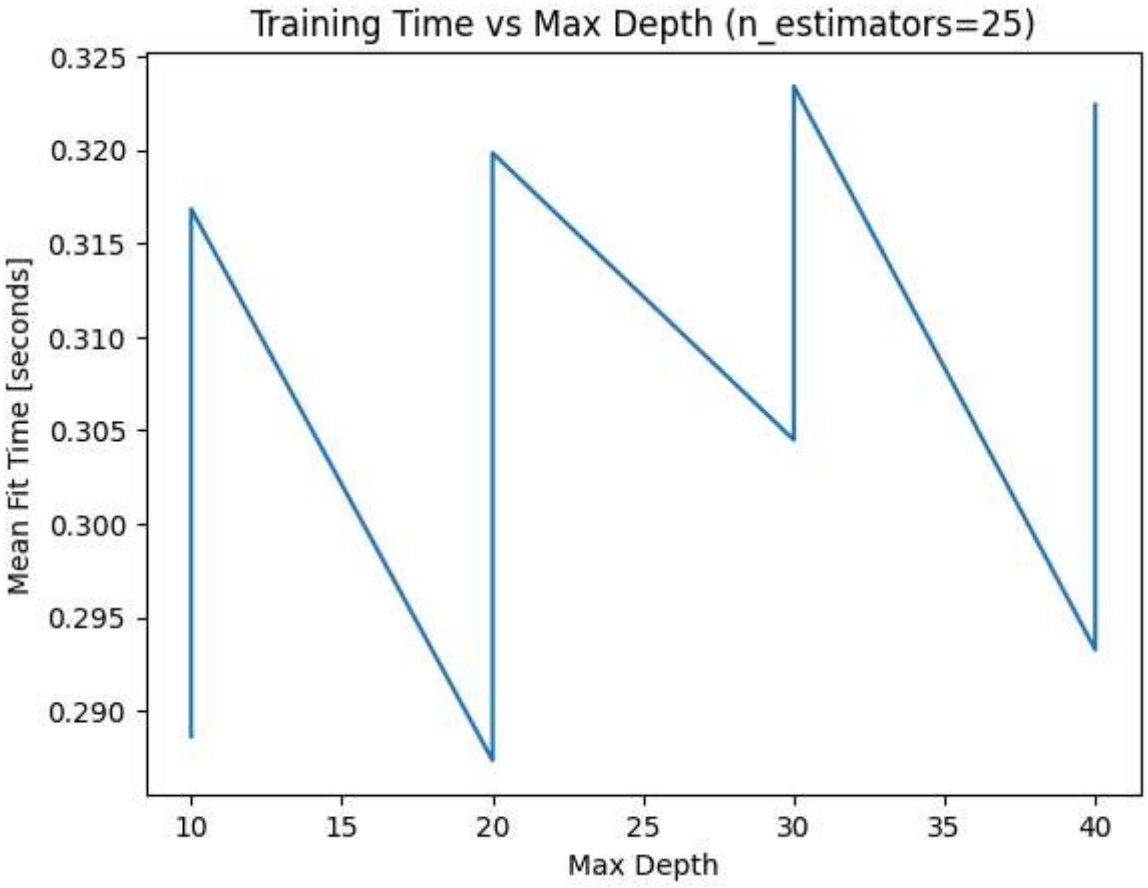
Out[66]:

05:04

Task 5.3.13: Create a mask for `cv_results` for rows where `"param_randomforestclassifier__n_estimators"` equals 25. Then plot `"param_randomforestclassifier__max_depth"` on the x-axis and `"mean_fit_time"` on the y-axis. Don't forget to label your axes and include a title.

- Subset a DataFrame with a mask using pandas.
- Create a line plot in Matplotlib.

In [68]:
```
# Create mask

mask = cv_results["param_randomforestclassifier__n_estimators"] == 25
# Plot fit time vs max_depth
plt.plot(
    cv_results[mask]["param_randomforestclassifier__max_depth"],
    cv_results[mask]["mean_fit_time"]
)

# Label axes
plt.xlabel("Max Depth")
plt.ylabel("Mean Fit Time [seconds]")
plt.title("Training Time vs Max Depth (n_estimators=25)");
```



There's a general upwards trend, but we see a lot of up-and-down here. That's because for each max depth, grid search tries two different imputation strategies: mean and median. Median is a lot faster to calculate, so that speeds up training time.

Finally, let's look at the hyperparameters that led to the best performance.

In [69]:
```
VimeoVideo("694695505", h="f98f660ce1", width=600)
```

Out[69]:

03:14

Task 5.3.14: Extract the best hyperparameters from `model`.

- Get settings from a hyperparameter search in scikit-learn.

In [70]:
```
# Extract best hyperparameters
model.predict(X_train_over)
```

Out[70]:
```
array([False, False, False, ...,  True,  True,  True])
```

Note that we don't need to build and train a new model with these settings. Now that the grid search is complete, when we use `model.predict()`, it will serve up predictions using the best model — something that we'll do at the end of this lesson.

Evaluate

All right: The moment of truth. Let's see how our model performs.

Task 5.3.15: Calculate the training and test accuracy scores for `model`.

- Calculate the accuracy score for a model in scikit-learn.

In [72]:
```
best_model = model.best_estimator_
```

In [73]:
```
from sklearn.metrics import accuracy_score

acc_train = accuracy_score(y_train, best_model.predict(X_train))
acc_test = accuracy_score(y_test, best_model.predict(X_test))

print("Training Accuracy:", round(acc_train, 4))
print("Test Accuracy:", round(acc_test, 4))
```

```
Training Accuracy: 1.0
Test Accuracy: 0.9989
```

We beat the baseline! Just barely, but we beat it.

Next, we're going to use a confusion matrix to see how our model performs. To better understand the values we'll see in the matrix, let's first count how many observations in our test set belong to the positive and negative classes.

In [74]:
```
y_test.value_counts()
```

Out[74]:
```
False    907
True      25
Name: bankrupt, dtype: int64
```

In [75]:
```
VimeoVideo("694695486", h="1d6ac2bf77", width=600)
```

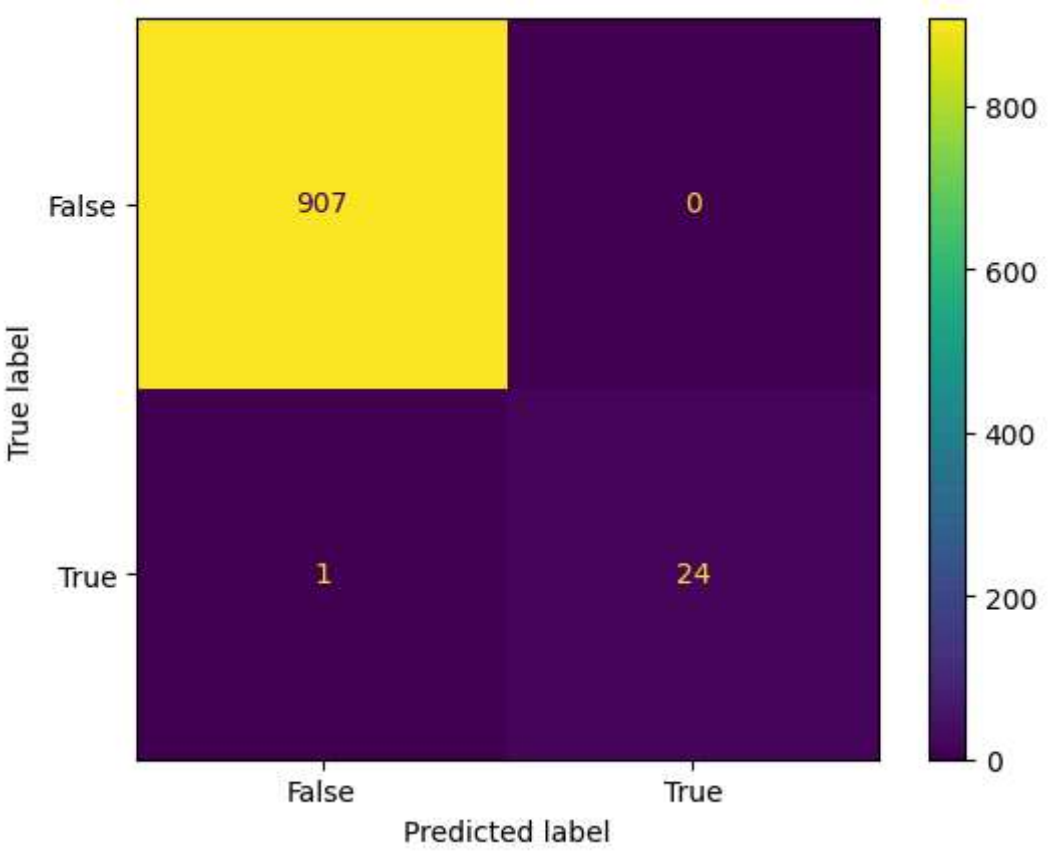
Out[75]:

04:04

Task 5.3.16: Plot a confusion matrix that shows how your best model performs on your test set.

- What's a confusion matrix?
- Create a confusion matrix using scikit-learn.

In [77]:
```
# Plot confusion matrix
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test);
```



Notice the relationship between the numbers in this matrix with the count you did the previous task. If you sum the values in the bottom row, you get the total number of positive observations in `y_test` ($72 + 11 = 83$). And the top row sum to the number of negative observations ($1903 + 10 = 1913$).

Communicate

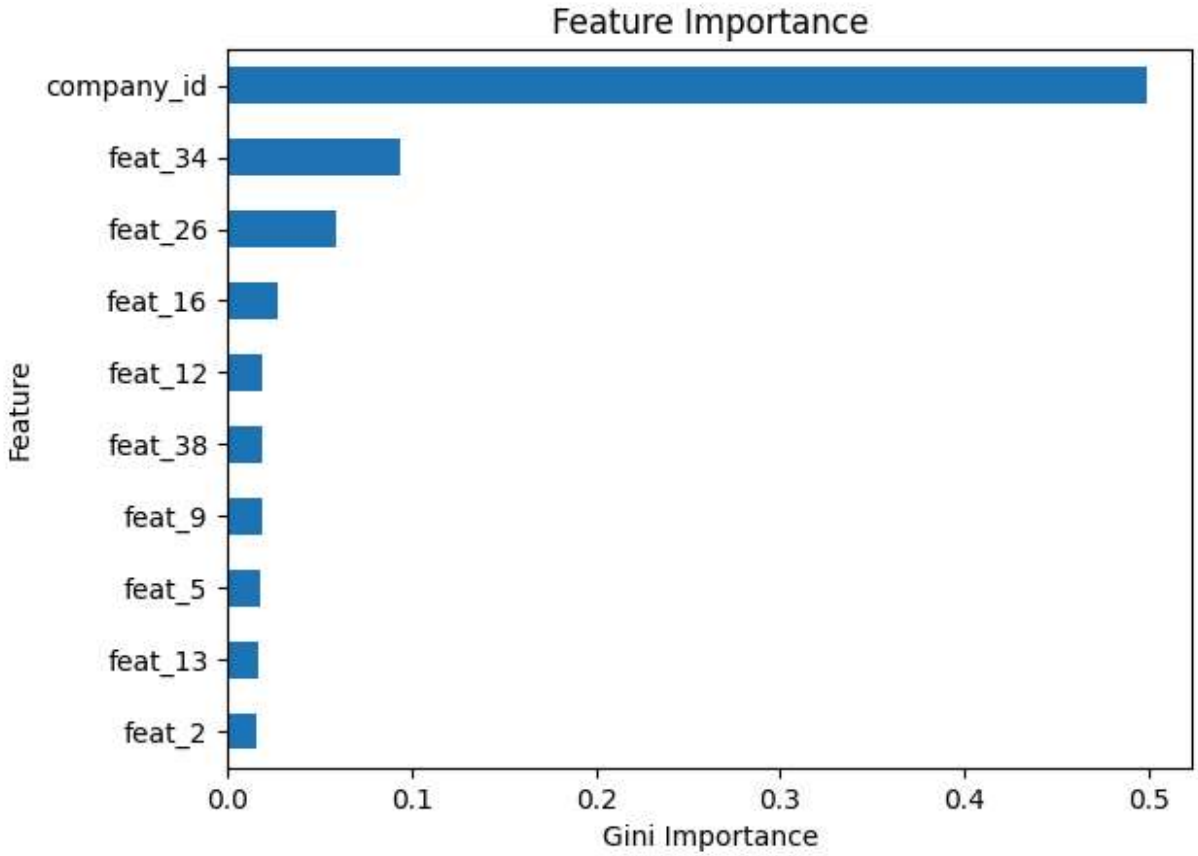
In [78]:
```
VimeoVideo("698358615", h="3fd4b2186a", width=600)
```

Out[78]:

05:39

Task 5.3.17: Create a horizontal bar chart with the 10 most important features for your model.

In [79]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Get feature names from training data
features = X_train.columns

# Extract importances from model
importances = best_model.named_steps["randomforestclassifier"].feature_importances_

# Create a series with feature names and importances
feat_imp = pd.Series(importances, index=features).sort_values(ascending=False)

# Plot 10 most important features
feat_imp[:10].sort_values().plot(kind="barh")

plt.xlabel("Gini Importance")
plt.ylabel("Feature")
plt.title("Feature Importance");
```



In [80]:
```
# # Get feature names from training data
# features = ...
# # Extract importances from model
# importances = ...
# # Create a series with feature names and importances
# feat_imp = ...
# # Plot 10 most important features


# plt.xlabel("Gini Importance")
# plt.ylabel("Feature")
# plt.title("Feature Importance");
```

The only thing left now is to save your model so that it can be reused.

In [ ]:
```
VimeoVideo("694695478", h="a13bdacb55", width=600)
```

Task 5.3.18: Using a context manager, save your best-performing model to a a file named `"model-5-3.pkl"`.

- What's serialization?
- Store a Python object as a serialized file using pickle.

In [81]:
```
# Save model
import pickle
```

```
with open("model-5-3.pkl", "wb") as f:
    pickle.dump(best_model, f)
```

In [ ]:
```
VimeoVideo("694695451", h="fc96dd8d1f", width=600)
```

Task 5.3.19: Create a function `make_predictions`. It should take two arguments: the path of a JSON file that contains test data and the path of a serialized model. The function should load and clean the data using the `wrangle` function you created, load the model, generate an array of predictions, and convert that array into a Series. (The Series should have the name `"bankrupt"` and the same index labels as the test data.) Finally, the function should return its predictions as a Series.

- What's a function?
- Load a serialized file
- What's a Series?
- Create a Series in pandas

In [82]:
```
# def make_predictions(data_filepath, model_filepath):
#     # Wrangle JSON file
#     X_test = ...
#     # Load model

#     # Generate predictions
#     y_test_pred = ...
#     # Put predictions into Series with name "bankrupt", and same index as X_test
#     y_test_pred = ...
#     return y_test_pred
```

In [83]:
```
import pickle
import pandas as pd

def make_predictions(data_filepath, model_filepath):
    # Wrangle JSON file
    X_test = wrangle(data_filepath)

    # Load model
    with open(model_filepath, "rb") as f:
        model = pickle.load(f)

    # Generate predictions
    y_test_pred = model.predict(X_test)

    # Put predictions into Series with name "bankrupt", and same index as X_test
    y_test_pred = pd.Series(y_test_pred, index=X_test.index, name="bankrupt")

    return y_test_pred
```

In [84]:
```
VimeoVideo("694695426", h="f75588d43a", width=600)
```

Out[84]:

04:00

Task 5.3.20: Use the code below to check your `make_predictions` function. Once you're satisfied with the result, click on the check the activity button on the left side of the panel.

In [85]:
```
y_test_pred = make_predictions(
    data_filepath="data/poland-bankruptcy-data-2009-mvp-features.json.gz",
    model_filepath="model-5-3.pkl",
)

print("predictions shape:", y_test_pred.shape)
y_test_pred.head()
```

```
predictions shape: (227,)
```

Out[85]:
```
0     False
3     False
7     False
8     False
18    False
Name: bankrupt, dtype: bool
```
